

Phase 2: Project Structure Generation

1. System Analysis

System Type: Web App

Features:

- User Authentication
- Course Management
- Content Delivery
- Grading and Feedback Mechanism
- Email Notifications
- Progress Tracking and Reporting

Non-Functional Requirements:

- Performance (2 seconds response time under normal load)
- Security (secure data storage and protection)
- Usability (intuitive and user-friendly interface)
- Reliability (99.9% uptime)
- Maintainability
- Portability (compatibility with various web browsers and devices)

Complexity: Medium

2. Architecture

Style: Layered Architecture

Pattern: Model-View-Controller (MVC)

Justification:

Given the medium complexity of the system and the specified features, a layered architecture is most appropriate as it promotes modularity, separation of concerns, and simplifies development. The MVC design pattern will help in implementing the user-friendly interface required by providing clear separation between data (Model), user interaction (View), and application logic (Controller). Additionally, this architecture style allows for good performance under normal load, security, reliability, maintainability, and portability.

3. Tech Stack

Frontend: React.js or Angular.js (for a more complex SPA structure)

Backend: Node.js with Express.js for its non-blocking, event-driven architecture

Database: MongoDB (NoSQL) for scalability and flexibility in handling dynamic data structures

Tools:

- Webpack for bundling assets, Babel for transpiling JavaScript, Jest for testing

Justification:

React.js or Angular.js are popular choices for building user-friendly interfaces, with a strong focus on performance and maintainability. Node.js with Express.js is suitable for the backend due to its scalable nature, high throughput, and compatibility with JavaScript throughout the stack. MongoDB as a NoSQL database offers flexibility in handling dynamic data structures, ease of scaling, and fast querying performance. Webpack, Babel, and Jest are commonly used tools that enhance

development efficiency and maintainability.

4. Project Structure

```

...
/my-project
|-- src
|   |-- backend
|       |-- controllers
|           |-- authController.js
|           |-- courseController.js
|           |-- contentController.js
|           |-- feedbackController.js
|           |-- emailController.js
|           |-- progressController.js
|       |-- models
|           |-- User.js
|           |-- Course.js
|           |-- Content.js
|           |-- Feedback.js
|           |-- Progress.js
|       |-- routes
|           |-- authRoutes.js
|           |-- courseRoutes.js
|           |-- contentRoutes.js
|           |-- feedbackRoutes.js
|           |-- emailRoutes.js
|           |-- progressRoutes.js
|   |-- app.js
|-- frontend
|   |-- src
|       |-- components
|           |-- Auth
|               |-- Login.js
|               |-- Register.js
|           |-- Course
|               |-- AddCourse.js
|               |-- ManageCourses.js
|           |-- Content
|               |-- DeliverContent.js
|           |-- Feedback
|               |-- GiveFeedback.js
|           |-- Progress
|               |-- TrackProgress.js
|       |-- App.js
|       |-- index.js
|-- package.json
...
```